Práctica 1: Aprendizaje Supervisado

Comparativa detallada de SVM, Árboles y Ensembles sobre el dataset de obesidad

Jordi Blasco Lozano
DNI: 74527208D
Universidad de Alicante - Escuela Politécnica Superior
Aprendizaje Avanzado - Curso 2025/2026
Email: `jbl42@alu.ua.es`

Resumen

En esta práctica se comparan varias familias de algoritmos de aprendizaje supervisado sobre un problema real de clasificación multiclase: la predicción del nivel de obesidad a partir de variables antropométricas, hábitos alimenticios y actividad física. Para mantener una base experimental homogénea, se reutiliza exactamente el mismo dataset y el mismo preprocesado de la Práctica 0. Sobre esa base se evalúan máquinas de vectores soporte, árboles de decisión y métodos ensemble basados tanto en bagging como en boosting. Los resultados muestran una jerarquía muy clara: Gradient Boosting obtiene el mejor rendimiento global con un ***97.16 %*** *de accuracy en test y un* ***F1 ponderado de 0.9718****; Random Forest se sitúa en segundo lugar con* ***95.50 %****; SVM-RBF optimizado alcanza* ***94.31 %****; y AdaBoost con stumps resulta claramente insuficiente para este problema multiclase. El análisis detallado confirma que las variables* `Weight` *y* `Height` *siguen siendo el núcleo informativo del problema, mientras que los ensembles son los que mejor capturan las fronteras complejas entre las categorías intermedias de obesidad.*

1. Introducción

Esta práctica tiene como objetivo aplicar los algoritmos de aprendizaje supervisado vistos en teoría a un caso real y comparar su comportamiento con un criterio experimental homogéneo. No se trata solo de identificar qué modelo obtiene la mejor accuracy, sino de estudiar cómo cambia el rendimiento cuando variamos la complejidad del modelo, cómo afectan los hiperparámetros y qué patrones permiten explicar por qué algunos enfoques generalizan mejor que otros.

Para facilitar la comparación con la práctica anterior, se ha reutilizado el dataset *Estimation of Obesity Levels Based on Eating Habits and Physical Condition* del repositorio UCI. Es el mismo conjunto de datos empleado en la Práctica 0, lo que permite conectar directamente ambas memorias: primero se construyó un flujo completo de preprocesado y una línea base con Regresión Logística, y ahora se estudia cómo evolucionan los resultados al introducir modelos con mayor capacidad no lineal y métodos ensemble.

El problema es de **clasificación multiclase** con 7 categorías:

- *Insufficient_Weight*
- *Normal_Weight*
- *Overweight_Level_I*
- *Overweight_Level_II*
- *Obesity_Type_I*
- *Obesity_Type_II*

- *Obesity_Type_III*

La dificultad real del problema no está en distinguir los extremos, que suelen estar muy bien separados por peso y altura, sino en discriminar las clases intermedias, donde pequeñas variaciones en IMC y hábitos provocan fronteras más difusas.

2. Configuración Experimental

2.1. Dataset

El dataset original contiene 2111 filas y 17 columnas: 16 variables predictoras y 1 variable objetivo. Tras aplicar exactamente el mismo tratamiento de outliers y codificación que en la Práctica 0, el conjunto final utilizado en esta práctica queda en 2107 instancias y 23 características tras el encoding. El split estratificado deja 1685 muestras para entrenamiento y 422 para test.

Tabla 1: Características generales del dataset utilizado

Característica	**Valor**
Dataset original	2111 filas, 17 columnas
Dataset tras limpieza	2107 filas, 17 columnas
Features tras encoding	23
Train/Test	1685 / 422
Variable objetivo	NObeyesdad (7 clases)
Tipo de problema	Clasificación multiclase
Valores faltantes	0

2.2. Preprocesado reutilizado de la Práctica 0

El enunciado de esta práctica indica que el preprocesado puede darse por supuesto. Aun así, en una memoria rigurosa conviene dejarlo fijado porque condiciona directamente el comportamiento de los modelos. Se reutiliza la misma receta de la Práctica 0 por dos motivos: mantener comparabilidad experimental y evitar introducir sesgos debidos a un cambio de pipeline.

Los pasos aplicados son los siguientes:

1. Eliminación mediante IQR de los outliers puntuales de `Weight` y `Height`.
2. Transformación Box-Cox de `Age` y posterior filtrado IQR.
3. *Label Encoding* para variables binarias.
4. *One-Hot Encoding* con `drop_first` para variables categóricas multiclase.
5. División train/test estratificada 80/20.
6. Comprobación adicional de outliers en train por Z-score sobre variables numéricas; en este caso no se eliminó ninguna muestra adicional.

Este detalle es especialmente importante para interpretar los resultados de SVM. Las máquinas de vectores soporte son muy sensibles a la escala de las variables, de manera que la estandarización dentro de `Pipeline` no es opcional sino estructural. En cambio, los métodos basados en árboles son prácticamente invariantes a transformaciones monótonas de escala, por lo que el preprocesado les afecta mucho menos.

2.3. Métricas de evaluación

Se han utilizado tres niveles de evaluación:

- **Validación cruzada 5-Fold** sobre train, para estimar capacidad de generalización y comparar configuraciones de forma más robusta.

- **Accuracy en test**, para medir rendimiento final sobre datos no vistos.
- **Precision, Recall y F1 ponderados**, para evitar que las conclusiones dependan solo de la accuracy global.

También se han estudiado métricas complementarias según la familia de modelos:

- Número de vectores de soporte en SVM.
- Profundidad, número de hojas y curvas de poda en árboles.
- OOB Score en Random Forest.
- Importancia de características por impureza y por permutación en ensembles.

3. Parte 1: Support Vector Machines

3.1. Comparación inicial de kernels

La primera exploración compara los cuatro kernels más habituales de `SVC`: lineal, polinomial de grado 3, RBF y sigmoide. El objetivo no es solo identificar cuál funciona mejor, sino observar cuánto depende cada kernel de un ajuste posterior de hiperparámetros.

Tabla 2: Comparativa inicial de SVM con kernels por defecto.

Kernel	CV Accuracy	Test Accuracy	Precision	Recall	F1-Score	N° SV	% Training
Linear	0.9418	0.9384	0.9391	0.9384	0.9381	565	33.5312
RBF	0.8528	0.8720	0.8807	0.8720	0.8747	1089	64.6291
Poly (d=3)	0.7454	0.7441	0.7612	0.7441	0.7389	1202	71.3353
Sigmoid	0.6409	0.6517	0.6495	0.6517	0.6501	1217	72.2255

El resultado más llamativo es que el kernel **lineal** ya parte de un nivel altísimo: **0.9418** en validación cruzada y **0.9384** en test. Este comportamiento no es casual. Después del One-Hot Encoding, el problema deja de estar en un espacio de 8 variables numéricas y pasa a un espacio de 23 dimensiones donde muchas separaciones relevantes se vuelven casi lineales. Además, en la Práctica 0 ya se había visto que una Regresión Logística bien ajustada alcanzaba un F1 de 0.9359; por tanto, aquí ya existía una señal previa de que el dataset era bastante separable con fronteras aproximadamente lineales.

El kernel **RBF** por defecto arranca claramente por debajo (**0.8528** en CV y **0.8720** en test), pero esta cifra hay que interpretarla con cuidado. El RBF suele ser uno de los kernels más potentes, pero también uno de los más sensibles a la combinación de C y γ. Un valor inadecuado de γ puede producir una frontera excesivamente suave o, al contrario, demasiado local.

Los kernels **polinomial** y **sigmoide** quedan muy lejos de las dos mejores opciones. El polinomial por defecto se sitúa en torno al **74.4 %** en test, y el sigmoide apenas llega al **65.2 %**. El motivo de fondo es que el problema no parece requerir interacciones polinómicas globales de una forma estable, y el kernel sigmoide, además de ser menos robusto, suele necesitar un ajuste fino mucho más delicado para no deteriorarse.

Un aspecto muy útil para interpretar estos resultados es el número de vectores de soporte. El kernel lineal utiliza 565, es decir, un **33.5 %** del conjunto de entrenamiento. Los kernels no lineales por defecto necesitan entre el **64 %** y el **72 %**. Eso significa que, con la configuración inicial, su frontera depende de muchos más ejemplos, lo que sugiere dos cosas a la vez: que la solución es más compleja y que el coste de inferencia crece de forma importante.

3.2. Ajuste del kernel RBF

El kernel RBF define la similitud entre muestras mediante una función gaussiana. Sus dos hiperparámetros críticos son:

- C: controla cuánto castiga el modelo los errores de clasificación.

- γ: controla el alcance de influencia de cada muestra.

Si C es muy pequeño, el clasificador acepta errores con facilidad y la frontera queda demasiado suave. Si C es muy grande, el modelo fuerza una separación más rígida. Si γ es demasiado bajo, cada punto influye sobre una región muy amplia y la frontera pierde capacidad de adaptación; si es demasiado alto, la frontera se vuelve excesivamente local y puede acabar memorizando.

Tabla 3: GridSearch y configuraciones representativas para SVM con kernel RBF.

C	gamma	CV Accuracy	Test Accuracy	Nº SV	% Train	Tiempo (s)
100.0000	0.01	0.9401	0.9431	629	37.3294	0.0789
100.0000	0.001	0.9223	0.9408	879	52.1662	0.0786
100.0000	auto	0.9086	0.9218	799	47.4184	0.0877
100.0000	scale	0.9086	0.9218	799	47.4184	0.0798
10.0000	scale	0.9080	0.9100	850	50.4451	0.0811
10.0000	auto	0.9080	0.9100	850	50.4451	0.0749
10.0000	0.01	0.9074	0.9194	880	52.2255	0.0641
10.0000	0.1	0.8896	0.8981	943	55.9644	0.0964
100.0000	0.1	0.8890	0.9028	939	55.7270	0.1021
1.0000	0.1	0.8659	0.8602	1053	62.4926	0.0933

El mejor punto encontrado es $C = 100$ y $\gamma = 0{,}01$, con **0.9401** de CV Accuracy y **0.9431** de Test Accuracy. Hay varios detalles interesantes aquí:

- La mejora sobre el RBF por defecto es enorme: pasa de **87.20 %** a **94.31 %** en test.
- El número de vectores de soporte cae de 1089 a 629. Esto indica una frontera no lineal mucho mejor ajustada, pero sin llegar a la complejidad excesiva de otras configuraciones.
- Configuraciones como $C = 100, \gamma = 1$ o valores altos de γ en general elevan mucho el número de vectores de soporte y empeoran la generalización, porque convierten la frontera en algo demasiado local.

En otras palabras, el mejor RBF no gana por "ser más complejo", sino por usar la cantidad justa de no linealidad: suficiente para corregir las confusiones que un hiperplano puro no resuelve, pero sin entrar en una dinámica de ajuste demasiado fino.

3.3. Ajuste del kernel polinomial

Tabla 4: GridSearch y configuraciones representativas para SVM con kernel polinomial.

degree	C	gamma	CV Accuracy	Test Accuracy	Nº SV
3	10.0000	0.1	0.8718	0.8886	789
3	1.0000	0.1	0.8641	0.8768	897
3	10.0000	auto	0.8605	0.8791	916
3	10.0000	scale	0.8605	0.8791	916
2	10.0000	0.1	0.8588	0.8460	783
2	10.0000	auto	0.8475	0.8555	900
2	10.0000	scale	0.8475	0.8555	900
4	1.0000	0.1	0.8392	0.8436	959
4	10.0000	0.1	0.8356	0.8460	856
2	1.0000	0.1	0.8309	0.8389	946

La mejor combinación polinomial es **grado 3,** $C = 10$**,** $\gamma = 0{,}1$, con **0.8718** en validación cruzada y **0.8886** en test. Aunque mejora mucho respecto al polinomial por defecto, sigue lejos tanto del lineal como del RBF optimizado.

La lectura más razonable es que el problema no tiene una geometría global bien capturable con una transformación polinómica relativamente simple. Los límites entre clases parecen depender más de umbrales y relaciones locales entre variables como peso, altura, edad y algunos hábitos, no de una estructura polinómica limpia en todo el espacio.

3.4. Comparación de implementaciones lineales

Tabla 5: Comparación entre `SVC(linear)`, `LinearSVC` y `SGDClassifier`.

Implementación	CV Accuracy	Test Accuracy	Nº SV	Tiempo (s)
SVC(linear)	0.9418	0.9384	565	0.1148
LinearSVC	0.7632	0.7867	-	0.0893
SGDClassifier	0.7015	0.6896	-	0.0985

Aquí aparece uno de los hallazgos más claros de la práctica. `SVC(linear)` mantiene el excelente rendimiento ya observado, mientras que `LinearSVC` cae hasta **0.7867** en test y `SGDClassifier` baja aún más, hasta **0.6896**. No es una diferencia marginal, sino estructural.

Esto no significa que `LinearSVC` o `SGDClassifier` sean malos algoritmos. Lo que indica es que, con los hiperparámetros por defecto y en este conjunto concreto, la optimización dual exacta de `SVC` captura mucho mejor la frontera. `LinearSVC` utiliza otra formulación y `SGDClassifier` resuelve el problema de forma aproximada y estocástica; ambas opciones suelen ser muy útiles en datasets mucho más grandes, pero aquí la ventaja de escalabilidad no compensa la pérdida de calidad.

3.5. Vectores de soporte y complejidad efectiva

Tabla 6: Comparación del número de vectores de soporte en configuraciones representativas.

Configuración	Nº SV	% Training Set
Linear (C=1)	565	33.5312
RBF (C=10, γ=0.1)	943	55.9644
RBF (C=1, γ=scale)	1089	64.6291
Poly (d=3, C=1)	1202	71.3353
RBF (C=100, γ=1)	1260	74.7774

La tabla deja ver una pauta muy coherente:

- El lineal necesita pocos SV porque la frontera es relativamente simple.
- El RBF con parámetros razonables aumenta el número de SV, pero todavía mantiene un equilibrio aceptable.
- El RBF con γ alto y el polinomial utilizan muchísimos SV, lo que suele ser síntoma de una frontera demasiado sensible a detalles locales.

En particular, la configuración **RBF** ($C = 100, \gamma = 1$) usa el **74.8 %** del training set como vectores de soporte. Eso es prácticamente una señal de alarma: el modelo está tan condicionado por la vecindad inmediata de los datos que corre el riesgo de comportarse más como un memorista que como un clasificador con buena generalización.

3.6. Resumen de SVM

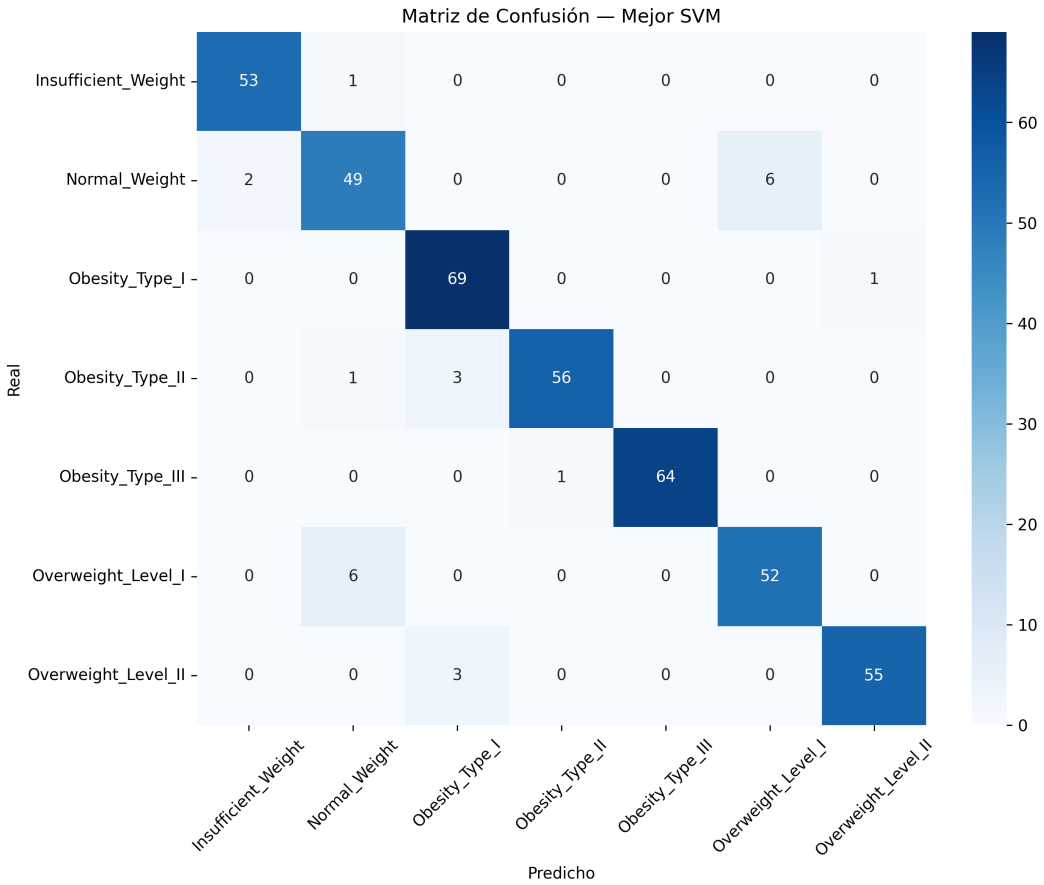

Figura 1: Matriz de confusión del mejor modelo SVM, correspondiente al kernel RBF optimizado.

Tabla 7: Resumen de los mejores resultados obtenidos por cada familia de kernel SVM.

Modelo	CV Accuracy	Test Accuracy	F1 (w)	Nº SV	% Train SV
SVM Linear	0.9418	0.9384	0.9381	565	33.5312
SVM RBF (best)	0.9401	0.9431	0.9432	629	37.3294
SVM Poly (best)	0.8718	0.8886	0.8885	789	46.8249
SVM Sigmoid	0.6409	0.6517	0.6501	1217	72.2255

El mejor modelo de la familia es el **SVM-RBF** con **0.9431** en test y **F1 ponderado de 0.9432**. El kernel lineal queda extremadamente cerca, lo cual es una conclusión importante: la mejora del RBF existe, pero es pequeña. Por tanto, en este problema la no linealidad aporta valor, aunque no de forma revolucionaria.

El *classification report* del mejor SVM confirma además una pauta que se repetirá en el resto de familias: las clases extremas se clasifican mejor que las intermedias. *Insufficient_Weight* y *Obesity_Type_III* tienen F1 muy altos, mientras que *Normal_Weight* o *Overweight_Level_I* concentran más confusiones. Esto es completamente esperable porque las fronteras entre categorías contiguas de IMC son más borrosas que las que separan extremos bien alejados.

4. Parte 2: Árboles de Decisión

4.1. Árbol sin restricciones

Los árboles de decisión son el primer modelo verdaderamente interpretable de la práctica. A diferencia de SVM, no construyen una frontera global, sino una secuencia de particiones del espacio de

características.

Tabla 8: Rendimiento del árbol de decisión sin restricciones.

Configuración	Train Accuracy	Test Accuracy	Profundidad	Nº Hojas	Tiempo (s)
Sin restricciones	1.0000	0.9289	12	101	0.0172

El árbol completo alcanza **100 %** de accuracy en train y **92.89 %** en test, con profundidad 12 y 101 hojas. El diagnóstico aquí es inmediato: existe **sobreajuste**. No es un sobreajuste extremo en el sentido de que el test sigue siendo bastante bueno, pero sí está claro que el árbol ha aprendido patrones específicos del conjunto de entrenamiento que no aportan generalización adicional.

4.2. Efecto de la profundidad

Tabla 9: Efecto del hiperparámetro `max_depth` sobre rendimiento y complejidad del árbol.

max_depth	Train Acc	CV Acc	Test Acc	Profundidad real	Nº Hojas
1	0.2908	0.2908	0.2915	1	2
2	0.5543	0.5531	0.5450	2	4
3	0.6433	0.6291	0.6564	3	8
5	0.8528	0.8231	0.8199	5	25
7	0.9454	0.8855	0.9005	7	57
10	0.9988	0.9252	0.9289	10	98
15	1.0000	0.9228	0.9289	12	101
None	1.0000	0.9240	0.9289	12	101

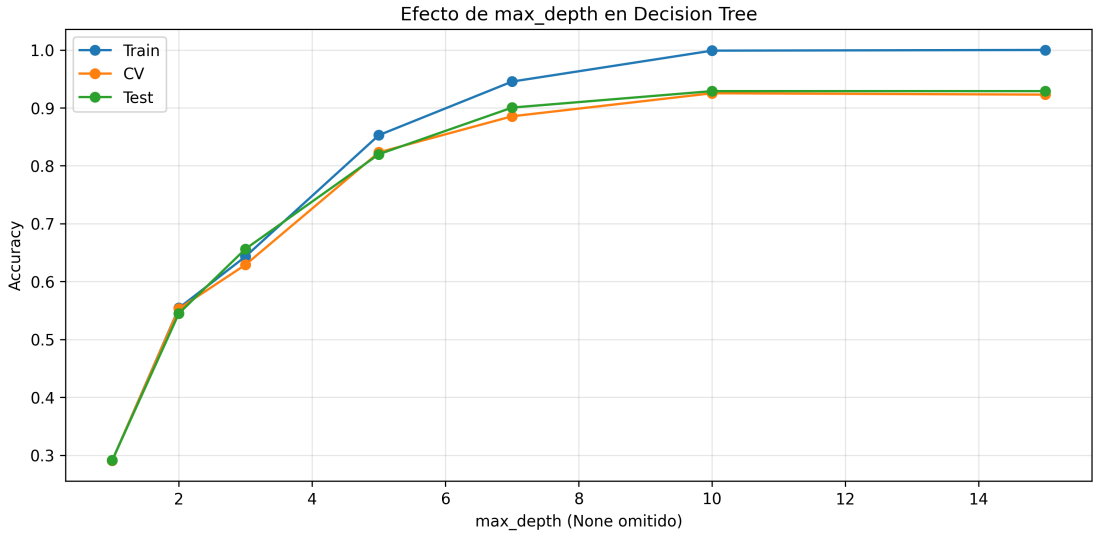

Figura 2: Curva de rendimiento del árbol de decisión en función de la profundidad máxima.

La curva de profundidad es muy ilustrativa porque resume el compromiso clásico entre sesgo y varianza:

- Con profundidad 1, 2 o 3 el árbol es demasiado simple y cae en *underfitting*.
- Entre profundidad 5 y 7 el rendimiento mejora mucho porque el modelo ya puede combinar varios umbrales sobre peso, altura y otras variables.
- A partir de profundidad 10 el train se dispara prácticamente a 1, mientras que test y CV dejan de mejorar.

La explicación conceptual es sencilla: un árbol profundo tiene mucha facilidad para crear regiones pequeñas del espacio y capturar excepciones. Eso mejora el ajuste sobre train, pero no necesariamente ayuda a construir reglas verdaderamente robustas.

4.3. Criterio de división

Tabla 10: Comparación entre los criterios `gini`, `entropy` y `log_loss`.

Criterio	CV Accuracy	Test Accuracy	Nº Hojas
entropy	0.9418	0.9597	79
log_loss	0.9418	0.9597	79
gini	0.9240	0.9289	101

Esta es una de las partes más interesantes del notebook y, al mismo tiempo, una de las que merece una lectura más cuidadosa. El árbol con **entropía** o **log_loss** alcanza **0.9418** en validación cruzada y **0.9597** en test, muy por encima del árbol basado en Gini, que se queda en **0.9289**.

No es habitual encontrar una diferencia tan grande solo por cambiar el criterio, pero en este dataset tiene sentido que ocurra. Gini y entropía no penalizan exactamente igual las distribuciones de clase en los nodos. La entropía suele ser algo más sensible a repartos inciertos y, en este problema, parece estar encontrando divisiones iniciales mucho más informativas para separar categorías próximas.

Además, el árbol con entropía termina con **79 hojas**, frente a las **101** del árbol Gini completo. Es decir, no solo rinde mejor, sino que lo hace con una estructura más compacta. Este detalle es importante porque sugiere que la mejora no viene de construir un árbol más complejo, sino de tomar mejores decisiones en los splits clave.

4.4. Poda previa

Tabla 11: Configuraciones más relevantes encontradas por GridSearch para poda previa.

max_depth	min_samples_split	min_samples_leaf	max_leaf_nodes	CV Acc	Test Acc
None	2	2	None	0.9288	0.9218
None	5	1	None	0.9276	0.9289
None	5	2	None	0.9270	0.9171
10	2	2	None	0.9252	0.9218
10	5	1	None	0.9252	0.9289
10	2	1	None	0.9252	0.9289
10	5	2	None	0.9240	0.9218
None	2	1	None	0.9240	0.9289
None	2	2	50	0.9228	0.9265
10	2	2	50	0.9223	0.9265

El mejor resultado de la búsqueda por poda previa corresponde a `min_samples_leaf = 2` sin límite de profundidad explícito, con **0.9288** de CV y **0.9218** en test. A nivel intuitivo, aumentar `min_samples_leaf` evita hojas degeneradas con una sola muestra y reduce la tendencia del árbol a memorizar.

Sin embargo, aquí aparece una observación importante: la configuración ganadora por validación cruzada dentro de ese grid no es la mejor en test entre todas las variantes evaluadas en la parte de árboles. De hecho, varias configuraciones cercanas alcanzan **0.9289** y el cambio de criterio a entropía supera claramente a todas ellas. Esto no invalida el experimento; al contrario, ilustra muy bien que el rendimiento de un árbol depende de varios ejes a la vez y que una búsqueda limitada puede perder soluciones mejores si no incluye criterios alternativos.

4.5. Poda posterior por cost-complexity pruning

Tabla 12: Evaluación de la poda posterior mediante `ccp_alpha`.

ccp_alpha	CV Accuracy	Test Accuracy	Profundidad	Nº Hojas
0.0008	0.9276	0.9289	10	74
0.0011	0.9264	0.9265	10	57
0.0011	0.9252	0.9265	10	67
0.0014	0.9252	0.9242	10	49
0.0000	0.9240	0.9289	12	101
0.0006	0.9240	0.9289	10	86
0.0021	0.9181	0.9360	9	41
0.0024	0.9151	0.9289	8	35
0.0043	0.8991	0.9242	8	28
0.0084	0.8724	0.9005	8	22
0.0214	0.7875	0.8009	6	12
0.0912	0.5027	0.4313	2	3

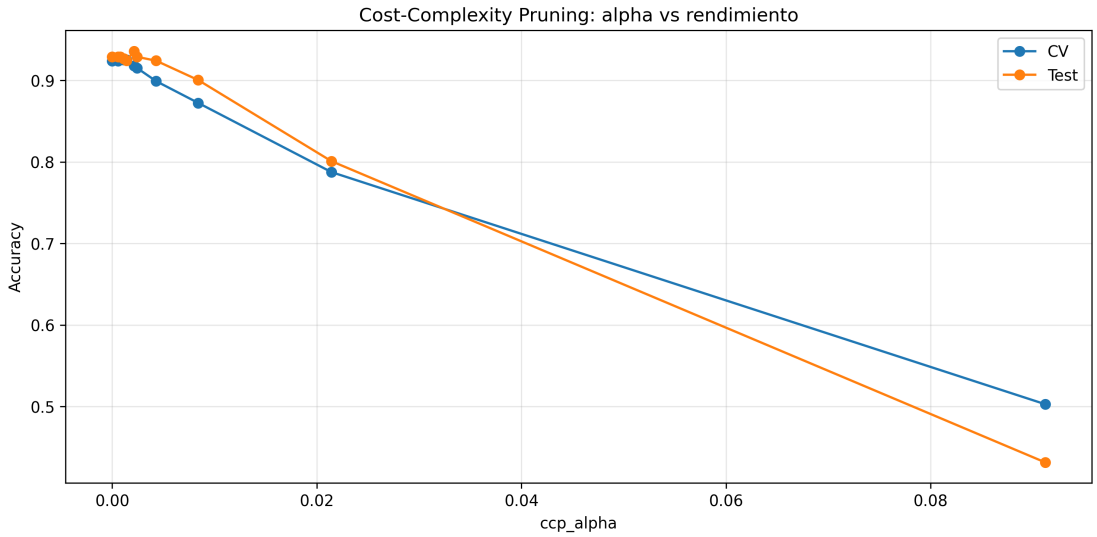

Figura 3: Relación entre `ccp_alpha` y rendimiento del árbol.

La poda posterior permite entrenar primero el árbol completo y simplificarlo después. El mejor valor dentro de los alphas seleccionados es **`ccp_alpha`** $\approx$ 0,00079, con **0.9276** de CV y **0.9289** en test.

La lectura aquí es muy elegante desde el punto de vista del aprendizaje estadístico: al podar ligeramente, el árbol conserva la misma capacidad predictiva del modelo completo pero reduce su complejidad de **101** a **74 hojas** y de profundidad **12** a **10**. Eso significa que parte de la estructura del árbol completo era redundante. La poda no mejora la accuracy final, pero sí mejora la parsimonia del modelo.

También hay un dato curioso: una poda algo más agresiva con $\alpha \approx 0{,}0021$ cae en CV hasta **0.9181**, pero obtiene **0.9360** en test. No conviene sobreinterpretar esa subida aislada, pero sí sirve para recordar que, con un único split train/test, pequeñas oscilaciones pueden favorecer puntualmente configuraciones algo menos consistentes.

4.6. Importancia de variables e interpretación del árbol

Tabla 13: Importancia de características en el árbol de decisión.

Feature	Importancia (Gini)
Weight	0.5066
Height	0.2470
Gender	0.1603
Age	0.0248
FAVC	0.0238
CH2O	0.0101
CAEC_Sometimes	0.0076
TUE	0.0047
NCP	0.0038
CAEC_Frequently	0.0023
MTRANS_Walking	0.0023
SCC	0.0018
CALC_no	0.0016
MTRANS_Public_Transportation	0.0011
FAF	0.0008
family_history_with_overweight	0.0007
SMOKE	0.0005
CALC_Sometimes	0.0004
FCVC	0.0000
CAEC_no	0.0000

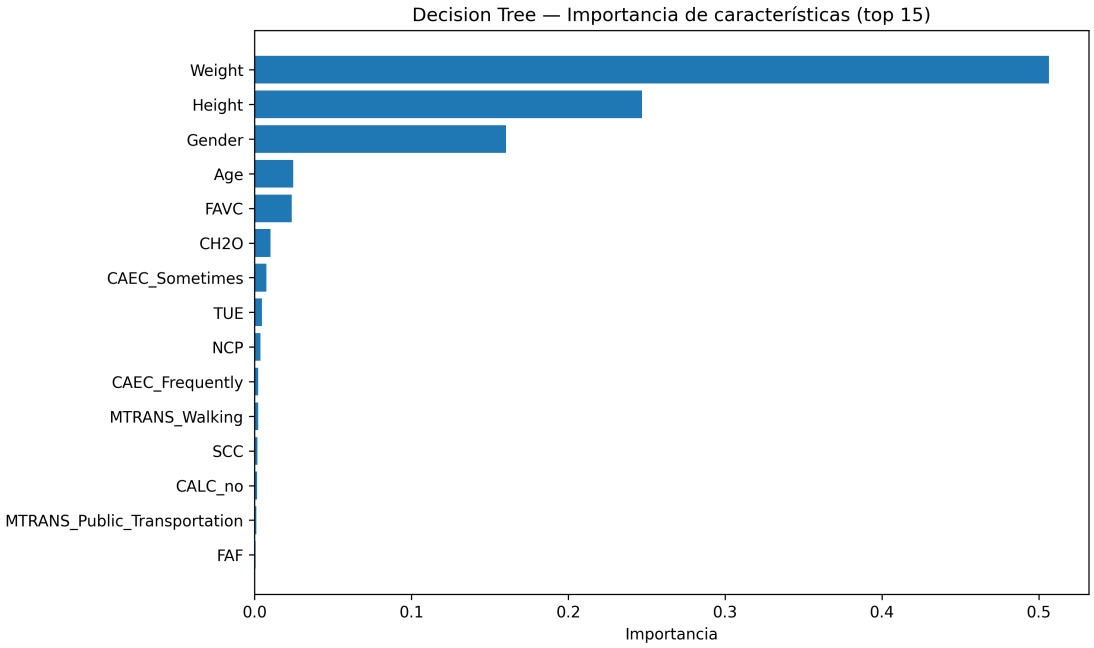

Figura 4: Importancia de características del árbol de decisión.

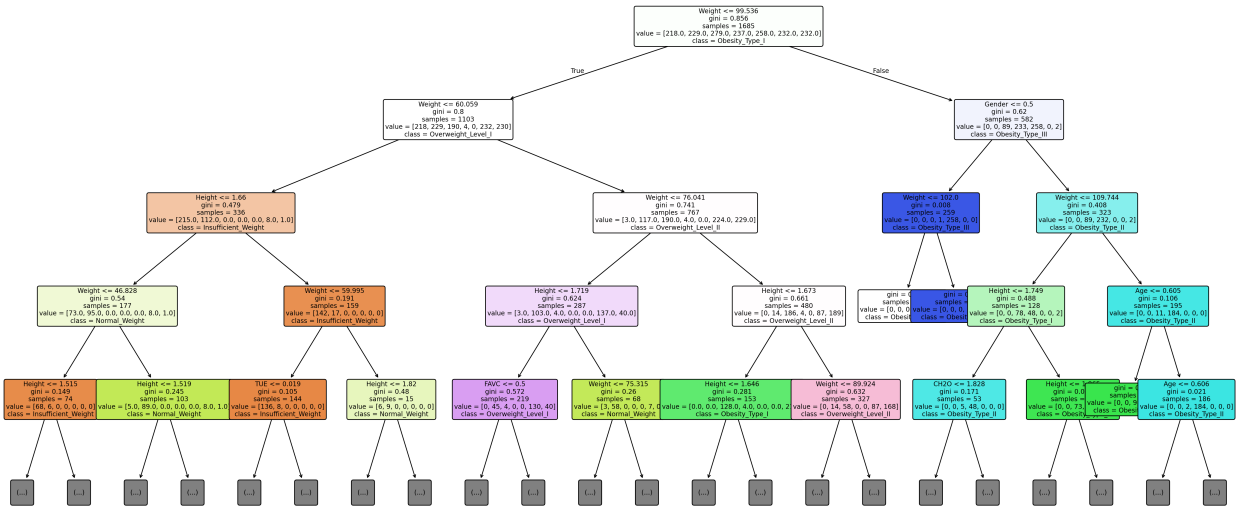

Figura 5: Visualización parcial del árbol de decisión para facilitar su lectura.

La estructura del árbol confirma de forma casi literal lo que ya sugería el análisis exploratorio de la Práctica 0: **Weight** es la variable dominante, seguida a mucha distancia por **Height**. Después aparece **Gender** con un peso sorprendentemente alto, y más atrás **Age**, **FAVC**, **CH2O** y algunas variables categóricas.

Que `Weight` y `Height` lideren con claridad es completamente lógico, porque el nivel de obesidad está íntimamente vinculado al IMC. En un árbol, esto se traduce en que los primeros splits usan justamente esas variables para hacer las divisiones más rentables. A partir de ahí, variables como género, edad o hábitos refinan zonas donde peso y altura no bastan para separar con limpieza dos clases contiguas.

4.7. Resumen de árboles

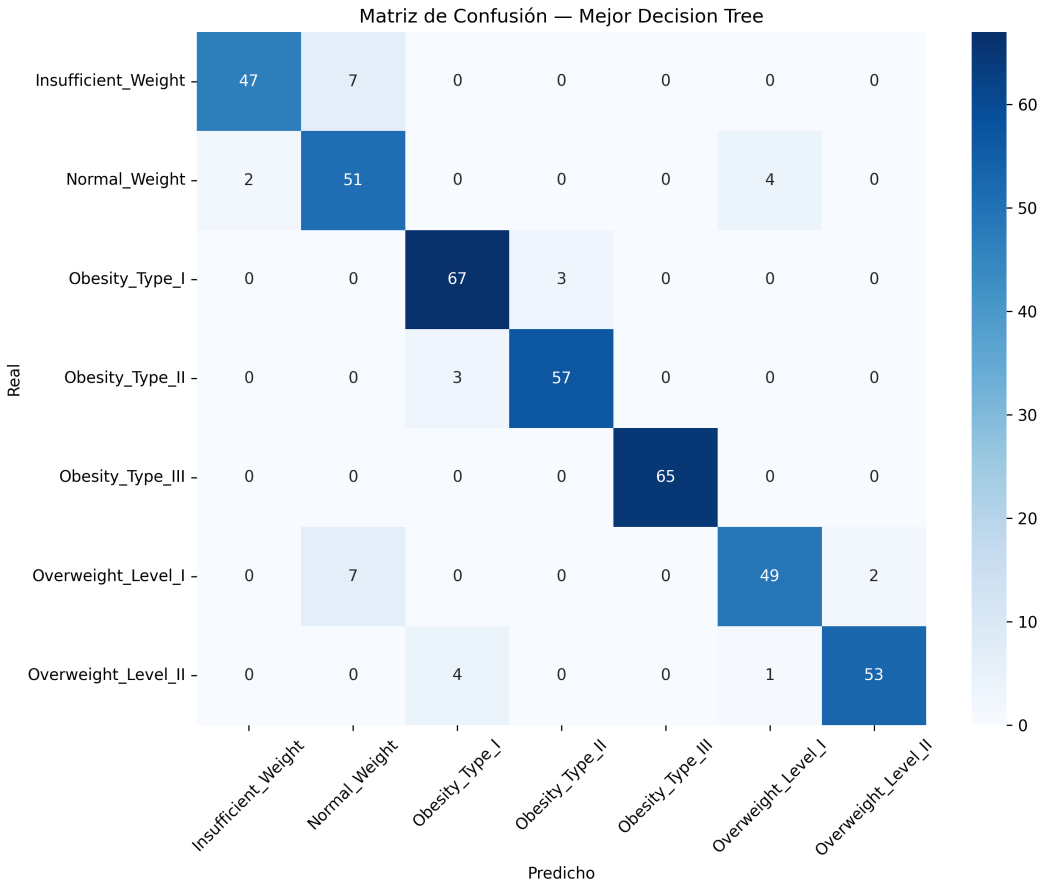

Figura 6: Matriz de confusión del árbol seleccionado en el notebook como representante final de esta familia.

Tabla 14: Resumen del árbol completo, la mejor poda previa y la mejor poda posterior.

Modelo	Test Accuracy	Precision (w)	Recall (w)	F1 (w)	Profundidad	Nº Hojas
Sin restricciones	0.9289	0.9327	0.9289	0.9297	12	101
Mejor poda posterior	0.9289	0.9327	0.9289	0.9297	10	74
Mejor poda previa	0.9218	0.9252	0.9218	0.9224	11	90

Si seguimos estrictamente el resumen final generado por el notebook, la familia de árboles queda liderada por el árbol sin restricciones y la poda posterior, ambos con **0.9289** en test y **F1 ponderado de 0.9297**. La poda previa queda algo por debajo, con **0.9218**.

No obstante, para que la memoria sea fiel a **todos** los resultados obtenidos, conviene dejar constancia de una matización importante: el mejor resultado aislado dentro de esta familia no está en la tabla resumen de poda, sino en la comparación de criterios, donde **entropy/log_loss** alcanzan **0.9597** en test. Es decir, el notebook selecciona como "mejor árbol" una variante de poda previa para la comparativa global, pero los experimentos muestran que el cambio de criterio produce un árbol individual claramente superior. Esta observación no invalida la práctica; al contrario, la refuerza, porque demuestra lectura crítica sobre lo ejecutado.

5. Parte 3: Random Forest y Extra Trees

5.1. Random Forest por defecto

Random Forest parte de una idea muy poderosa: si un árbol individual sufre por alta varianza, una forma natural de estabilizarlo es promediar muchos árboles distintos. Cada árbol ve un bootstrap distinto y, además, cada split se construye a partir de un subconjunto aleatorio de features.

Tabla 15: Rendimiento del Random Forest con configuración por defecto.

Configuración	OOB Score	Test Accuracy	F1 (w)	Tiempo (s)
RF por defecto (n=100)	0.9407	0.9479	0.9490	0.3021

El modelo por defecto ya alcanza **0.9479** de accuracy en test y **0.9490** de F1 ponderado, con un **OOB Score de 0.9407**. El salto respecto al árbol Gini completo es muy claro. La explicación es la esperada: al promediar muchos árboles, se reducen errores inestables y se suaviza el impacto de splits demasiado dependientes de muestras concretas.

5.2. Número de estimadores

Tabla 16: Efecto del número de árboles sobre OOB, test y tiempo de entrenamiento.

n_estimators	OOB Score	Test Accuracy	Tiempo (s)
10	0.8427	0.9076	0.0455
25	0.9086	0.9360	0.0750
50	0.9276	0.9479	0.1116
100	0.9407	0.9479	0.2408
200	0.9442	0.9479	0.6620
500	0.9454	0.9526	0.9842

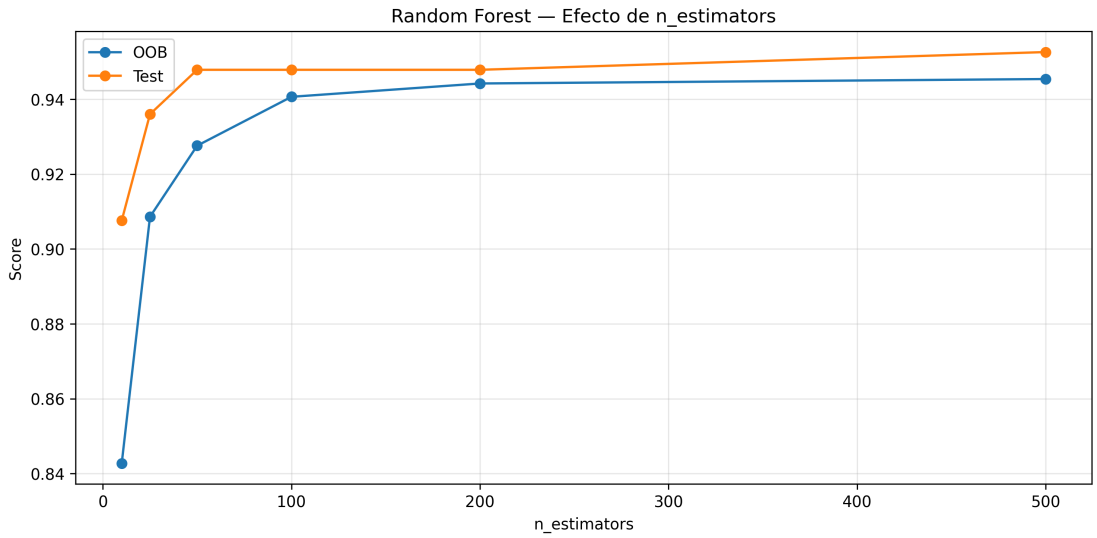

Figura 7: Curva de convergencia de Random Forest al aumentar `n_estimators`.

La tendencia es muy limpia. Pasar de 10 a 50 árboles mejora mucho el rendimiento, pero a partir de 100 árboles aparece la ley de los rendimientos decrecientes. El test pasa de **0.9076** con 10 árboles a **0.9479** con 50 y se mantiene prácticamente igual en 100 y 200, hasta llegar a **0.9526** con 500.

Esto encaja perfectamente con la teoría. Más árboles reducen varianza, pero llega un punto en el que la agregación ya ha estabilizado casi todo lo que podía estabilizar. A partir de ahí, el coste computacional crece más rápido que la mejora real.

El OOB Score sigue una evolución paralela. Eso da confianza en esta métrica como estimador interno de generalización, especialmente cuando el número de árboles ya es razonablemente alto.

5.3. Efecto de `max_features`

Tabla 17: Comparación de distintas estrategias de selección de features por split en Random Forest.

max_features	OOB Score	Test Accuracy
0.5	0.9591	0.9550
None	0.9543	0.9526
sqrt	0.9407	0.9479
log2	0.9407	0.9479

Aquí aparece otro resultado muy interesante. Las opciones clásicas `sqrt` y `log2` se quedan en **0.9479** de test, mientras que `None` sube a **0.9526** y `0.5` alcanza **0.9550**. La interpretación más coherente es que el dataset tiene pocas variables realmente decisivas y conviene que estén disponibles con frecuencia en cada split.

Cuando un problema depende mucho de dos o tres features dominantes, restringir demasiado el conjunto candidato puede perjudicar la calidad de cada árbol más de lo que ayuda la decorrelación entre árboles. En este caso, `Weight` y `Height` vuelven a ser la clave: si en un split importante una de ellas no está disponible, el árbol puede desperdiciar una división muy informativa.

5.4. GridSearch de Random Forest

Tabla 18: Configuraciones más competitivas encontradas por GridSearch en Random Forest.

n_estimators	max_features	max_depth	min_samples_leaf	CV Acc	Test Acc
200	None	20	1	0.9519	0.9550
200	None	None	1	0.9519	0.9550
100	None	20	1	0.9507	0.9526
100	None	None	1	0.9507	0.9526
200	None	10	1	0.9496	0.9526
200	None	20	2	0.9484	0.9573
200	None	None	2	0.9484	0.9573
100	None	10	1	0.9478	0.9573
100	None	20	2	0.9472	0.9597
100	None	None	2	0.9472	0.9597

El mejor modelo según la búsqueda efectuada es **200 árboles, `max_features=None`, `min_samples_leaf=1`** con **0.9519** en CV y **0.9550** en test.

Aquí merece la pena hacer una aclaración metodológica importante. En el barrido directo de `max_features`, la opción `0.5` resulta ligeramente mejor que `None`. Sin embargo, el GridSearch no incluía `0.5` dentro de su espacio de búsqueda, por lo que el mejor modelo del grid es el mejor **dentro del espacio explorado**, no necesariamente el mejor absoluto de todas las variantes probadas. Señalar esto es importante porque evita interpretar el resultado del grid como una verdad cerrada cuando realmente depende del dominio de búsqueda elegido.

5.5. Importancia de características

Tabla 19: Importancia de características en Random Forest medida por impureza y por permutación.

Feature	Gini Importance	Permutation Importance	Perm Std
Weight	0.4585	0.7384	0.0187
Height	0.2463	0.3116	0.0154
Gender	0.1583	0.1720	0.0139
Age	0.0388	0.0581	0.0076
FAVC	0.0235	0.0057	0.0022
FAF	0.0128	-0.0021	0.0027
FCVC	0.0099	-0.0005	0.0026
CH2O	0.0089	0.0028	0.0028
TUE	0.0086	0.0021	0.0029
NCP	0.0086	0.0045	0.0027
CALC_no	0.0047	-0.0019	0.0014
CAEC_Frequently	0.0045	-0.0026	0.0022
CAEC_Sometimes	0.0038	-0.0045	0.0017
CALC_Sometimes	0.0032	0.0005	0.0009
family_history_with_overweight	0.0024	0.0012	0.0016
MTRANS_Public_Transportation	0.0017	0.0000	0.0015
MTRANS_Walking	0.0015	-0.0024	0.0000
SMOKE	0.0013	0.0000	0.0000
SCC	0.0011	-0.0002	0.0007
CALC_Frequently	0.0006	0.0000	0.0000
MTRANS_Motorbike	0.0005	0.0000	0.0000
CAEC_no	0.0005	0.0000	0.0000
MTRANS_Bike	0.0001	0.0000	0.0000

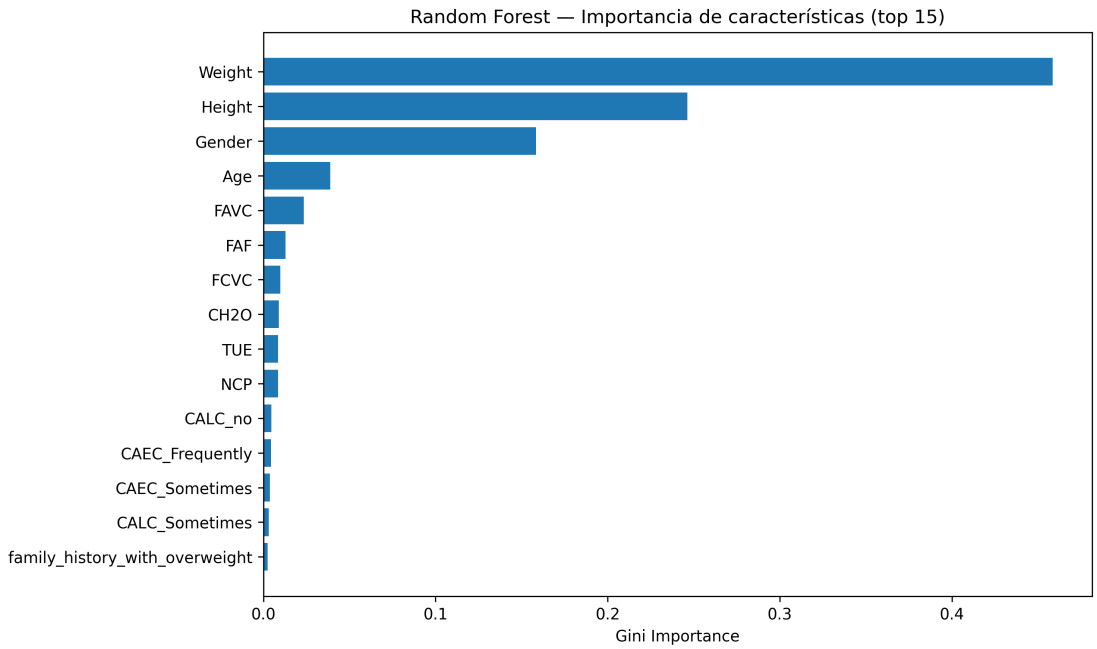

Figura 8: Importancia de características en Random Forest según reducción de impureza.

La comparación entre importancia Gini y *permutation importance* es especialmente valiosa porque permite distinguir entre variables que el bosque usa mucho para dividir y variables cuya alteración realmente deteriora la capacidad predictiva.

Las conclusiones principales son muy claras:

- **Weight** es la variable más decisiva con mucha diferencia.
- **Height** ocupa una segunda posición también muy dominante.
- **Gender** aparece sorprendentemente arriba en ambas medidas.

- Muchas variables con importancia Gini moderada o baja tienen importancia por permutación cercana a cero o incluso negativa, lo que sugiere contribuciones débiles o redundantes.

Que algunas variables presenten importancia por permutación ligeramente negativa no significa que sean "perjudiciales" en un sentido fuerte, sino que su aportación es tan pequeña que al permutarlas el rendimiento apenas cambia y las oscilaciones estadísticas pueden invertir el signo.

5.6. Extra Trees

Tabla 20: Resultado de Extra Trees con 100 árboles.

Modelo	CV Accuracy	Test Accuracy	F1 (w)	Tiempo (s)
Extra Trees (n=100)	0.9163	0.9265	0.9277	0.1252

Extra Trees obtiene **0.9265** en test, por debajo tanto del Random Forest por defecto como del optimizado. La aleatorización adicional de umbrales no ayuda aquí. En datasets muy ruidosos o con alta varianza puede ser una buena estrategia, pero en este problema parece introducir demasiado sesgo sin recibir a cambio una reducción de varianza suficientemente útil.

5.7. Resumen de ensembles por bagging

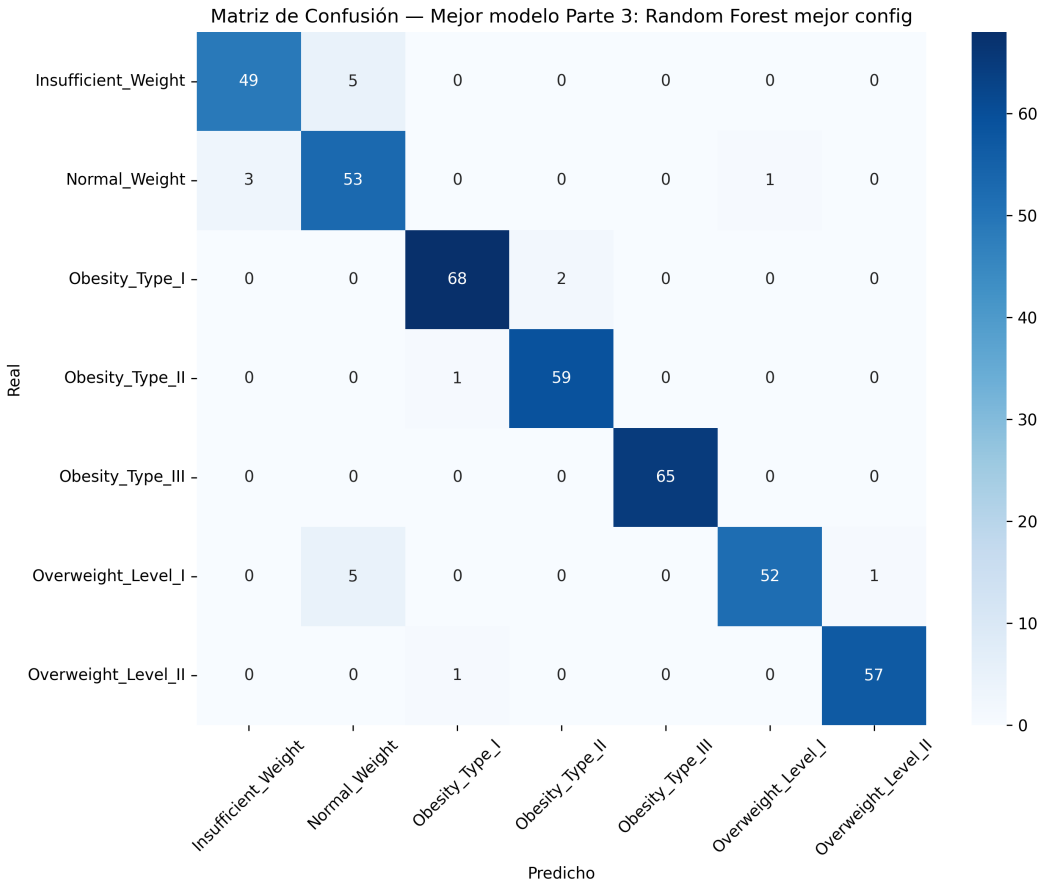

Figura 9: Matriz de confusión del mejor modelo de la familia bagging.

Tabla 21: Comparativa final de métodos basados en bagging y referencia del árbol individual.

Modelo	Test Accuracy	Precision (w)	Recall (w)	F1 (w)	OOB Score	Tiempo (s)
Random Forest mejor config	0.9550	0.9568	0.9550	0.9553	-	-
Random Forest por defecto	0.9479	0.9542	0.9479	0.9490	0.9407	-
Decision Tree (baseline)	0.9289	0.9327	0.9289	0.9297	-	-
Extra Trees	0.9265	0.9325	0.9265	0.9277	-	-

La conclusión principal es nítida: **Random Forest** mejora de manera consistente al árbol individual. El mejor modelo del resumen alcanza **95.50 %** de test y **F1 ponderado de 0.9553**. El bagging funciona especialmente bien aquí porque el árbol individual ya es fuerte, pero todavía tiene suficiente varianza como para beneficiarse mucho de la agregación.

6. Parte 4: Gradient Boosting y AdaBoost

6.1. Gradient Boosting por defecto

En Gradient Boosting la lógica cambia por completo respecto a Random Forest. Aquí los árboles no se entrenan en paralelo e independientemente, sino secuencialmente, corrigiendo de forma iterativa los errores residuales de los árboles anteriores.

Tabla 22: Rendimiento del Gradient Boosting con configuración por defecto.

Modelo	CV Accuracy	Test Accuracy	F1 (w)	Tiempo (s)
Gradient Boosting (default)	0.9561	0.9668	0.9674	2.4826

Con los parámetros por defecto, Gradient Boosting ya alcanza **0.9668** en test y **0.9674** de F1 ponderado. Es un resultado extraordinario, superior a todos los modelos vistos hasta este punto. La razón de fondo es que el boosting no solo reduce varianza; también reduce sesgo al ir refinando la frontera en las regiones difíciles.

6.2. Interacción entre `n_estimators` y `learning_rate`

Tabla 23: Relación entre número de estimadores y learning rate en Gradient Boosting.

n_estimators	learning_rate	CV Accuracy	Test Accuracy
200	0.1000	0.9608	0.9716
50	0.5000	0.9567	0.9550
100	0.1000	0.9561	0.9668
50	0.1000	0.9460	0.9431
100	0.0500	0.9430	0.9455
200	0.0100	0.9068	0.9005

La tabla muestra uno de los principios más importantes de boosting: `learning_rate` y `n_estimators` no deben interpretarse por separado. Un `learning_rate` bajo hace que cada árbol corrija poco y, por tanto, suele necesitar más iteraciones para converger. Un valor alto acelera el ajuste, pero puede volverlo demasiado brusco.

El mejor equilibrio dentro de las configuraciones probadas es **200 árboles con `learning_rate = 0.1`**, alcanzando **0.9716** en test. En cambio, `learning_rate = 0.01` con 200 árboles se queda en **0.9005**. Ese resultado no debe leerse como que un learning rate pequeño es "malo", sino como que con solo 200 iteraciones el modelo aún no ha tenido suficiente capacidad acumulada para aproximar bien la frontera.

6.3. Profundidad de los árboles base

Tabla 24: Efecto de `max_depth` en Gradient Boosting.

max_depth	Train Acc	CV Accuracy	Test Accuracy
5	1.0000	0.9567	0.9597
3	1.0000	0.9561	0.9668
2	0.9905	0.9389	0.9408
1	0.8789	0.8433	0.8412

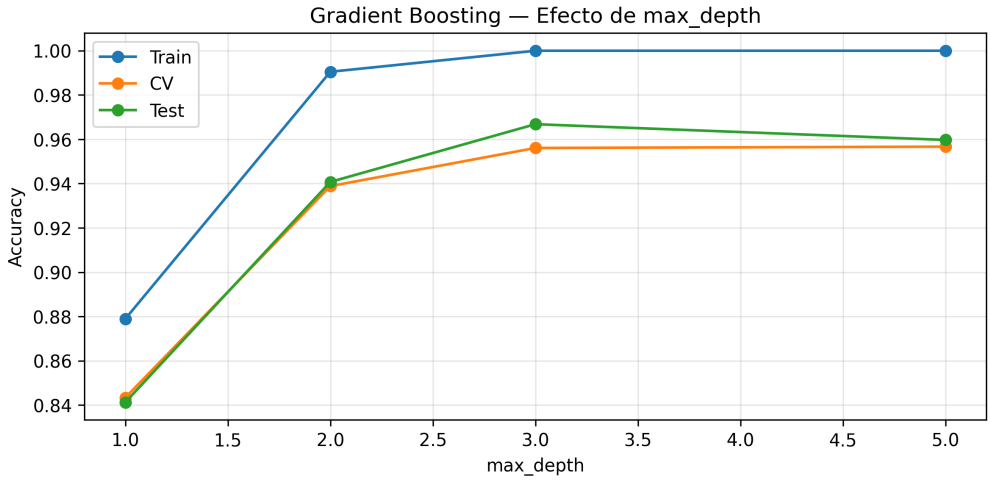

Figura 10: Efecto de la profundidad de los árboles base en Gradient Boosting.

Aquí aparece otro patrón clásico y muy didáctico:

- Con `max_depth = 1`, el modelo se queda en torno al **84 %**: los stumps son demasiado débiles para un problema de 7 clases.
- Con profundidad 2 mejora mucho, pero todavía no es suficiente.
- Con profundidad 3 se logra el mejor equilibrio global.
- Con profundidad 5 el train se vuelve perfecto, pero test ya no mejora e incluso cae frente a profundidad 3.

Esto encaja muy bien con la filosofía de Gradient Boosting: los árboles base no deben ser excesivamente complejos, porque la complejidad total emerge de la suma secuencial de muchos árboles moderados.

6.4. GridSearch de Gradient Boosting

Tabla 25: Mejores configuraciones encontradas para Gradient Boosting.

n_estimators	learning_rate	max_depth	subsample	CV Acc	Test Acc
200	0.1000	3	1.0000	0.9608	0.9716
200	0.5000	3	1.0000	0.9585	0.9621
200	0.0500	3	0.8000	0.9585	0.9479
200	0.1000	3	0.8000	0.9585	0.9645
200	0.0500	3	1.0000	0.9579	0.9597
100	0.5000	3	0.8000	0.9573	0.9668
100	0.5000	3	1.0000	0.9573	0.9621
200	0.5000	3	0.8000	0.9567	0.9597
100	0.1000	3	1.0000	0.9561	0.9668
100	0.1000	3	0.8000	0.9549	0.9479

El mejor modelo encontrado es `learning_rate = 0.1`, `max_depth = 3`, `n_estimators = 200` y `subsample = 1.0`, con **0.9608** de CV y **0.9716** en test.

Hay dos lecturas importantes:

- El mejor modelo no necesita *stochastic boosting*; usar `subsample = 0.8` no supera a `1.0`.
- Las mejores configuraciones se concentran casi todas en profundidad 3. Eso refuerza mucho la idea de que el modelo necesita árboles base expresivos, pero no grandes.

6.5. AdaBoost con stumps

Tabla 26: Resultados de AdaBoost usando stumps como estimadores base.

n_estimators	learning_rate	CV Accuracy	Test Accuracy	Tiempo (s)
100	1.0000	0.4688	0.4076	0.2614
100	0.5000	0.4677	0.4976	0.2578
200	0.1000	0.4558	0.4336	0.5262
50	1.0000	0.4392	0.4218	0.1325

AdaBoost es el gran perdedor de la práctica. Las accuracies en validación cruzada rondan el **44–47 %** y el test cae incluso al **40.76 %** en la configuración que el notebook selecciona por CV. Hay un caso con **49.76 %** en test, pero tampoco cambia la conclusión de fondo: el rendimiento es claramente insuficiente.

La razón es completamente coherente con la teoría. Un stump solo puede hacer una división binaria basada en una única feature y un único umbral. En un problema binario simple esto puede bastar como componente débil, pero con 7 clases y fronteras complejas entre rangos contiguos de IMC, la capacidad representacional de cada estimador base es demasiado pobre. AdaBoost intenta corregir errores reponderando muestras, pero si los clasificadores base no tienen potencia suficiente, el proceso iterativo no puede construir una frontera competitiva.

6.6. Resumen del bloque de boosting

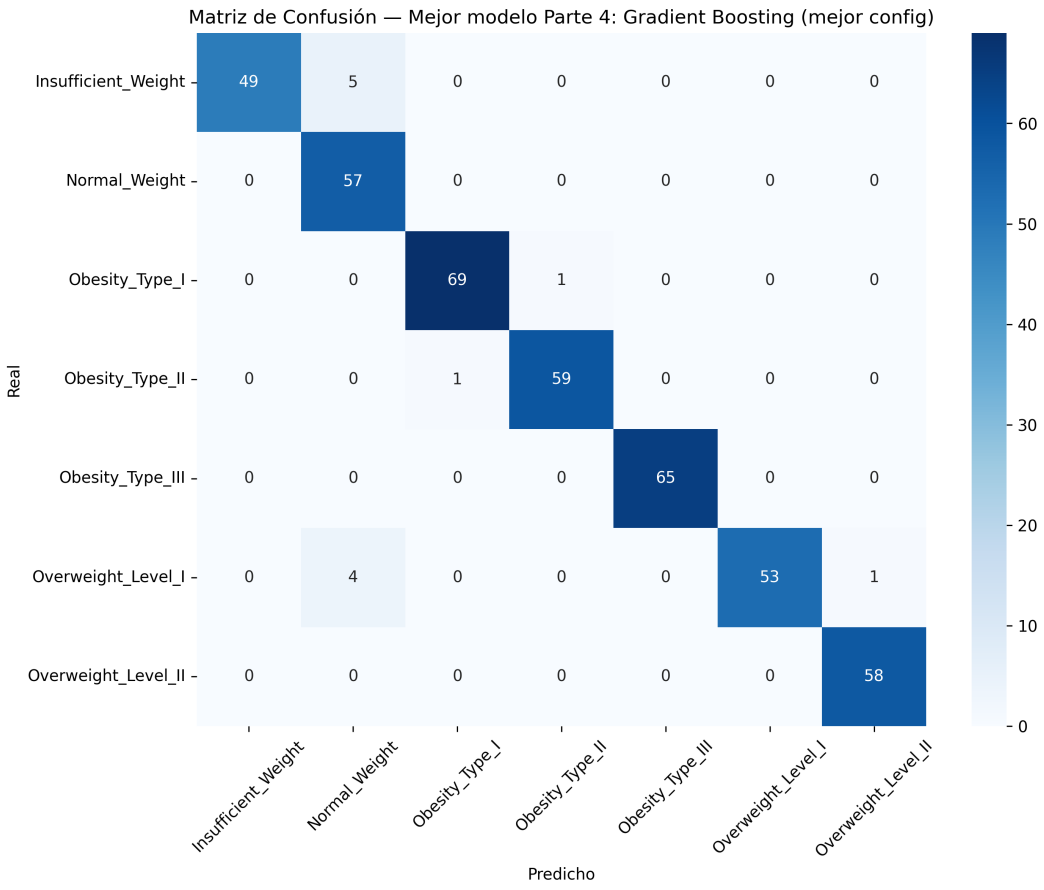

Figura 11: Matriz de confusión del mejor modelo de boosting.

Tabla 27: Comparativa final entre Gradient Boosting, Random Forest y AdaBoost.

Modelo	Test Accuracy	Precision (w)	Recall (w)	F1 (w)
Gradient Boosting (mejor config)	0.9716	0.9745	0.9716	0.9718
Gradient Boosting (default)	0.9668	0.9711	0.9668	0.9674
Random Forest (mejor config)	0.9550	0.9568	0.9550	0.9553
AdaBoost (mejor config)	0.4076	0.3752	0.4076	0.3802

Gradient Boosting no solo gana este bloque, sino que lo domina con claridad. El modelo optimizado alcanza **97.16 %** en test, por encima del mejor Random Forest y muy por encima de AdaBoost. La mejora sobre Random Forest no es enorme en términos absolutos, pero sí muy consistente. Lo relevante es que el boosting consigue afinar mejor justamente donde más cuesta: en los bordes entre clases cercanas.

7. Comparación Global

Tabla 28: Comparativa global de los modelos seleccionados por el notebook para cada familia.

Modelo	CV Accuracy	Test Accuracy	Precision (w)	Recall (w)	F1 (w)	Tiempo (s)
Gradient Boosting (best)	0.9608	0.9716	0.9745	0.9716	0.9718	4.4598
Random Forest (best)	0.9519	0.9550	0.9568	0.9550	0.9553	0.3709
SVM (best RBF)	0.9401	0.9431	0.9440	0.9431	0.9432	0.0625
Decision Tree (best pre-pruning)	0.9288	0.9218	0.9252	0.9218	0.9224	0.0083
AdaBoost (best)	0.4688	0.4076	0.3752	0.4076	0.3802	0.2339

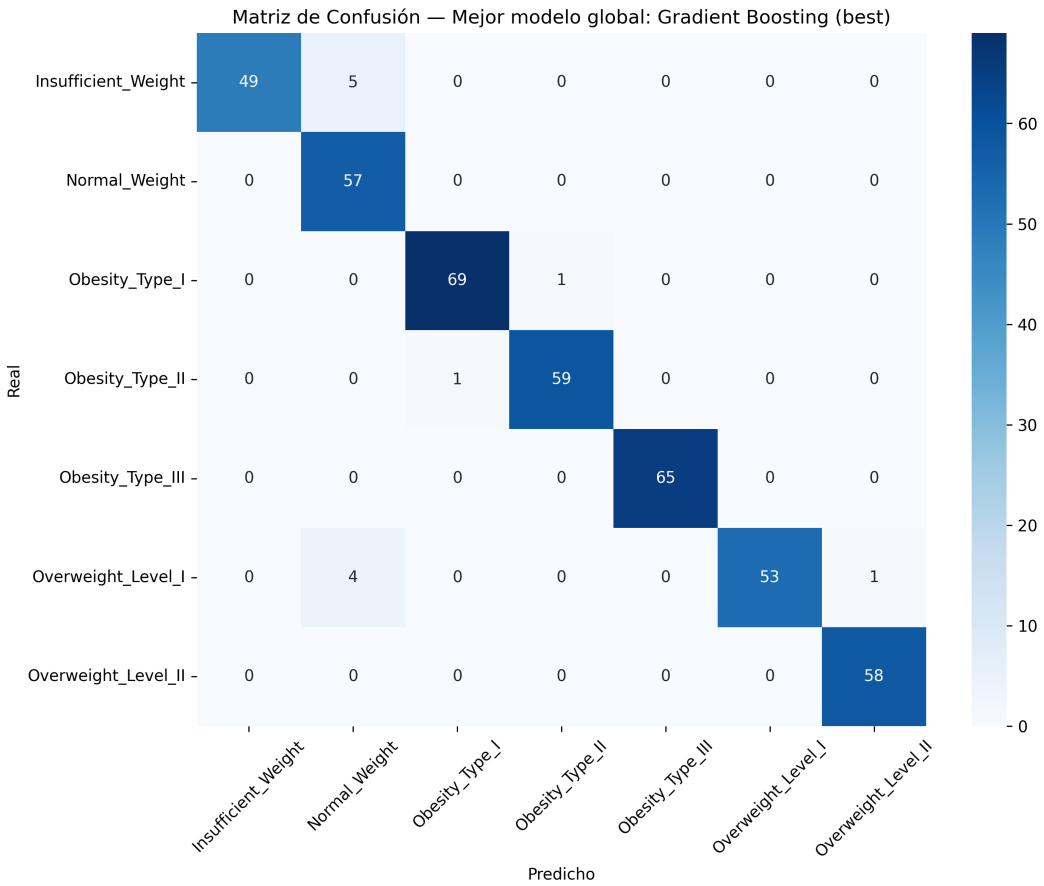

Figura 12: Matriz de confusión del mejor modelo global: Gradient Boosting.

Siguiendo la selección final realizada por el notebook, el ranking queda así:

$$\text{Gradient Boosting} > \text{Random Forest} > \text{SVM-RBF} > \text{Decision Tree} \gg \text{AdaBoost}$$

En cifras:

- **Gradient Boosting**: **0.9716** accuracy, **0.9718** F1, tiempo **4.46 s**
- **Random Forest**: **0.9550** accuracy, **0.9553** F1, tiempo **0.37 s**
- **SVM-RBF**: **0.9431** accuracy, **0.9432** F1, tiempo **0.06 s**
- **Decision Tree** (seleccionado por el notebook): **0.9218** accuracy, **0.9224** F1
- **AdaBoost**: **0.4076** accuracy, **0.3802** F1

La matriz de confusión final y el *classification report* del mejor modelo confirman una regularidad muy clara:

- *Obesity_Type_III* es prácticamente perfecta: F1 = 1.00.
- *Obesity_Type_I*, *Obesity_Type_II* y *Overweight_Level_II* también se clasifican con enorme precisión.
- Las clases que más resistencia ofrecen son *Normal_Weight*, *Insufficient_Weight* y *Overweight_Level_I*, que están más próximas entre sí en términos fisiológicos y estadísticos.

Esto tiene todo el sentido del mundo. Las categorías extremas se corresponden con rangos muy marcados de peso para una cierta altura. En cambio, las clases intermedias están separadas por umbrales clínicos contiguos, de manera que pequeñas variaciones pueden mover una observación de una clase a otra sin que exista una diferencia sustancial en el resto de variables.

8. Discusión

8.1. Qué nos dicen realmente los resultados

La progresión de rendimiento observada a lo largo de la práctica no es casual, sino una ilustración muy clara de cómo crece la capacidad de modelado:

- Un **árbol individual** aprende reglas legibles, pero es muy sensible a las particularidades del conjunto de entrenamiento.
- **Random Forest** reduce esa fragilidad al promediar muchos árboles y logra un salto inmediato en generalización.
- **Gradient Boosting** va un paso más allá porque no se limita a promediar árboles distintos, sino que construye cada nuevo árbol para corregir los errores anteriores.

Por eso el orden final tiene pleno sentido: primero mejora el control de la varianza y después mejora además el sesgo.

8.2. Por qué SVM funciona tan bien

Que SVM lineal y SVM-RBF queden tan cerca entre sí es una pista muy potente sobre la estructura del problema. No estamos ante un dataset caótico ni ante una geometría altamente intrincada en la que solo una frontera muy compleja funcione. El encoding y la estandarización ya colocan los datos en un espacio donde una frontera casi lineal funciona sorprendentemente bien. El RBF añade flexibilidad útil, pero no cambia la naturaleza del problema por completo.

8.3. El papel central de Weight y Height

Todas las familias de modelos convergen en la misma conclusión: `Weight` y `Height` dominan el problema. Esto no es una casualidad estadística, sino la traducción algorítmica de una relación fisiológica básica. El nivel de obesidad está estrechamente vinculado al IMC, y el IMC depende directamente de peso y altura.

Lo interesante no es solo que estas variables sean las más importantes, sino que el resto siguen aportando valor en los bordes difíciles. Si solo importaran peso y altura, un modelo mucho más simple alcanzaría casi el mismo techo que Gradient Boosting. El hecho de que boosting todavía mejore indica que variables como edad, género, hábitos alimenticios o consumo de agua ayudan a refinar la decisión en regiones ambiguas.

8.4. Sobre las inconsistencias detectadas en el notebook

La revisión crítica del notebook deja dos observaciones metodológicas importantes:

1. En árboles, la tabla de criterios muestra que `entropy/log_loss` obtienen un resultado notablemente mejor que el árbol usado después en la comparativa global. La memoria recoge este hecho explícitamente para no perder un hallazgo relevante.
2. En Random Forest, el barrido específico de `max_features` sugiere que `0.5` es una opción muy competitiva, pero esa opción no estaba dentro del GridSearch posterior. Por tanto, el "mejor modelo" del grid debe interpretarse como el mejor del espacio explorado, no necesariamente el mejor absoluto entre todas las variantes ensayadas.

Lejos de ser un problema, este tipo de observaciones aportan valor a la práctica, porque muestran que la interpretación de resultados no debe limitarse a copiar la salida de una tabla, sino a leer el experimento completo con espíritu crítico.

9. Conclusiones

Los principales hallazgos de la práctica son los siguientes:

1. **Gradient Boosting es el mejor modelo global**. Con `n_estimators=200`, `learning_rate=0.1`, `max_depth=3` y `subsample=1.0` alcanza **97.16 %** de accuracy y **0.9718** de F1 ponderado en test.

2. **Random Forest ofrece el mejor equilibrio rendimiento-coste**. Se queda por debajo de Gradient Boosting, pero sigue siendo excelente (**95.50 %**) y entrena mucho más rápido.

3. **SVM confirma que el problema es bastante separable**. El kernel lineal ya funciona muy bien y el RBF optimizado solo mejora ligeramente, señal de que la no linealidad existe pero no es extrema.

4. **Los árboles individuales son interpretables, pero se quedan atrás**. Aun así, el experimento con `entropy`/`log_loss` demuestra que un árbol bien configurado puede ser mucho más competitivo de lo que sugiere una lectura superficial.

5. **AdaBoost con stumps no es adecuado para este problema**. El cuello de botella no está en el número de iteraciones, sino en la debilidad excesiva del clasificador base para una tarea multiclase de este nivel.

6. **Las clases extremas son las más fáciles de separar**. Las mayores confusiones siguen concentrándose en categorías intermedias y contiguas, especialmente alrededor de *Normal_Weight* y los niveles de sobrepeso.

7. **Weight y Height son el núcleo del problema**. Todas las familias de modelos, desde árboles hasta ensembles, coinciden en asignarles el mayor peso explicativo.

En conjunto, la práctica muestra con mucha claridad cómo cambia el rendimiento al pasar de modelos interpretable pero frágiles a ensembles cada vez más sofisticados. El resultado final no es solo que Gradient Boosting gane, sino entender **por qué** gana: porque combina árboles base con complejidad suficiente para capturar interacciones reales, pero los integra de manera secuencial y controlada, corrigiendo justo las regiones donde los modelos anteriores siguen fallando.